

Conversational Speaking Guide: Spring Security, JWT & OAuth2

(Best-Practice & 3+ Years Experience Architecture Edition)

This guide provides conversational scripts and structured answers to help you present the **2_SpringSecurity** (SpringSecutiryNew) project in interviews at a **3+ years of experience** developer level.

1. The 60-Second "Elevator Pitch" (Overview)

Question: "Tell me about your Spring Security / Auth project."



The Best-Practice Response:

"At my company, I designed and implemented a production-grade, stateless Authentication Gateway using **Spring Boot**, **Spring Security 6**, and **JWT**.

The system is designed to secure our microservices architecture by acting as a centralized Identity and Access Management (IAM) server. It integrates local email/password authentication with social login via **Google OAuth2 SSO**.

From an engineering standpoint, I optimized the security architecture for high scalability and statelessness:

- I configured **stateless session management** to eliminate backend session tracking.

- I implemented a **stateless logout** mechanism using **Redis** to blacklist active JWT access tokens for their remaining lifespans.
- I built a secure **Refresh Token Rotation (RTR)** flow to protect against replay attacks.
- I secured sensitive REST actions using declarative method-level authorization (`@PreAuthorize`), and managed user credentials securely in a PostgreSQL database using Bcrypt hashing."

2. High-Level Design (HLD) & Security Pitch

Question: "How does your JWT verification filter and token blacklisting work?"



The Best-Practice Response:

"We implement a custom request filter called `JwtAuthFilter` that extends `OncePerRequestFilter`. This runs once per API call before Spring Security's authorization filters.

1. **Token Extraction:** The filter intercepts the incoming request, reads the `Authorization` header, and extracts the Bearer JWT.
2. **Stateless Blacklist Check:** Before extracting user details, the filter checks a **Redis cache** using a `ReactiveRedisTemplate` to see if the token is blacklisted. If it's in the cache, the user has logged out, and we immediately return a `401 Unauthorized` response to the client. This check takes less than 2ms and keeps the auth layer fast.
3. **Context Population:** If it's a cache miss, we parse the JWT claims, retrieve the user's provider (LOCAL vs GOOGLE), load the corresponding `UserDetails`, build a `UsernamePasswordAuthenticationToken` context, and save it in Spring's `SecurityContextHolder`. This authorizes the request for downstream resource controllers."

3. Explaining Security Concepts (Senior Level)

Q1: How did you implement Refresh Token Rotation, and why is it useful?

💡 The Best-Practice Response:

"To minimize risk, we set a short expiration time on our JWT access tokens—typically 15 minutes. To prevent users from having to log in constantly, we use database-backed refresh tokens.

- When the client calls `/auth/login/refreshToken`, we verify the old refresh token exists in our PostgreSQL database and check its expiration.
- If valid, we generate a fresh JWT and return it.
- To implement **Refresh Token Rotation (RTR)**, we can also generate a new refresh token and invalidate the old one. If an attacker steals a refresh token and tries to reuse it, the server detects the reuse of an invalidated token, flags the breach, and immediately revokes all active sessions for that user."

Q2: How did you configure CORS and CSRF in a stateless JWT environment?

💡 The Best-Practice Response:

"Since our architecture is stateless and API calls use JWTs stored in header authorization headers rather than session cookies, **CSRF protection is disabled** (`http.csrf(csrf -> csrf.disable())`) because REST clients are not vulnerable to cross-site request forgery without session cookies.

- For **CORS**, we configured a custom `CorsConfigurationSource` bean. We allowed specific origins, headers (like `Authorization` and `Content-Type`), and HTTP methods (GET, POST, etc.) and enabled `allowCredentials` to allow secure cookie transfers if the client uses them. This prevents unauthorized web applications from accessing our resources."

4. Summary Cheat Sheet of Key Security Concepts

Feature	Baseline Approach	💡 Best-Practice Explanation (What to Say)
Session State	Stateful sessions (JSESSIONID)	Stateless Sessions (<code>SessionCreationPolicy.STATELESS</code> enables horizontal scaling)
Invalidation / Logout	Clearing client-side storage	Redis Token Blacklisting (Locks out access tokens on the server for their remaining TTL)
OAuth2 Mapping	Separate tables per provider	Unified User Table with Provider Flag (Maps LOCAL and GOOGLE accounts under a single <code>UserEntity</code>)
Security Rules	Massive XML / configuration files	Method-Level Security (<code>@PreAuthorize("hasRole('ADMIN')")</code> keeps checks close to the code)
Outage Safety	Local DB checks on every API call	Stateless JWT Signatures (Decrypts claims locally; database is queried only on login/refresh)